

FISHY!

Documentacion tecnica, funcional y operativa

CLIENTE	BACKEND	DATOS
Unity 6	Django REST	PostgreSQL y JSON

Edicion: 3 de septiembre de 2026

Fuente editable: README.md

Contenido

Este PDF se genera directamente desde el README del repositorio. Los numeros de pagina se actualizan al regenerar el documento.

Resumen	5
Objetivos del producto	5
Tecnologias principales	5
Estado actual	6
Arquitectura	6
Principios de integracion	6
Estructura del repositorio	7
Directorios que pueden causar confusion	7
Requisitos	7
Backend	7
Cliente Unity	7
Herramientas opcionales	8
Inicio rapido	8
1. Preparar el backend en Windows	8
2. Preparar el backend en Linux o macOS	8
3. Ejecutar sin Supabase	8
4. Abrir Unity	9
Configuracion del backend	9
Variables de entorno	9
Configuraciones Django	9
Autenticacion y permisos	9
Base de datos remota	10
Configuracion del proyecto Unity	10
Escenas	10
Scripts y modulos activos	10
Entrada e interaccion	10
Configurar la API en Unity	10
Flujos principales	11
Control parental e inicio de partida	11
Conversacion con NPC	11

Caso de detective	11
Progreso de mision	11
Sistemas del juego	12
Inventario y objetos recogibles	12
Misiones	12
Riesgo y zonas	12
Estado y expresiones de Otto	12
Telefono y chat	13
Banco de contenido	13
Banco de preguntas	13
Casos de detective	13
Fuente de verdad y sincronizacion	13
Modelo de datos	14
Identidad y juego	14
Conversaciones	14
Detective	14
Misiones y album	14
Migraciones	14
API HTTP	15
Autenticacion	15
Perfiles infantiles	15
Partidas, NPC y riesgo	15
Chats y mensajes	16
Banco de preguntas	16
Detective	16
Codigos de respuesta esperados	16
Persistencia	17
Pruebas y validacion	17
Backend	17
Prueba de humo	17
Prueba global	17
Unity	18
Carga y administracion de datos	18
Cargar banco de preguntas y catalogos	18
Cargar casos de detective	18
Verificar el banco	18

Administrador Django	18
Postman	18
Despliegue y seguridad	19
Docker	19
Antes de produccion	19
Consideraciones sobre menores	19
Problemas conocidos	19
Solucion de problemas	20
El backend no inicia por conexion a PostgreSQL	20
/api/health/ no responde	20
Unity no llega al backend	20
Login devuelve 401	20
Error de indice al entrar al login o menu	20
El objeto se recoge, pero no aparece en el inventario	20
El banco de Unity y el backend muestran contenido distinto	20
Las migraciones no estan sincronizadas	21
Flujo de trabajo recomendado	21
Ramas	21
Cambios de backend	21
Cambios de contenido	21
Cambios de Unity	21
Documentacion complementaria	21
Mantenimiento de este documento	22

Documentacion tecnica, funcional y operativa del proyecto.

Estado documentado: 3 de septiembre de 2026. Rama base: `dev` (`42d3be2`).

Resumen

Fishy! es un videojuego educativo para enseñar seguridad digital a niños y niñas mediante exploración, conversaciones, decisiones, misiones y casos de detective. La experiencia combina:

- Un cliente desarrollado en Unity.
- Un backend REST desarrollado con Django y Django REST Framework.
- Una base de datos PostgreSQL alojada en Supabase para los entornos conectados.
- Bancos JSON versionados para preguntas, respuestas y casos de detective.
- Un flujo de control parental en el que un adulto crea y administra los perfiles infantiles.

El juego presenta situaciones relacionadas con desconocidos, privacidad, ciberacoso, exclusion y retos virales. Cada decisión puede cambiar el nivel de riesgo de la partida, el estado del personaje Otto y el progreso del jugador.

Objetivos del producto

- Entregar aprendizaje contextual, no solo preguntas aisladas.
- Registrar las decisiones importantes para que el progreso sea recuperable.
- Separar la cuenta del adulto de los perfiles infantiles.
- Permitir que el contenido educativo evolucione sin reescribir toda la lógica del juego.
- Mantener el cliente Unity desacoplado de la base de datos mediante una API HTTP.

Tecnologías principales

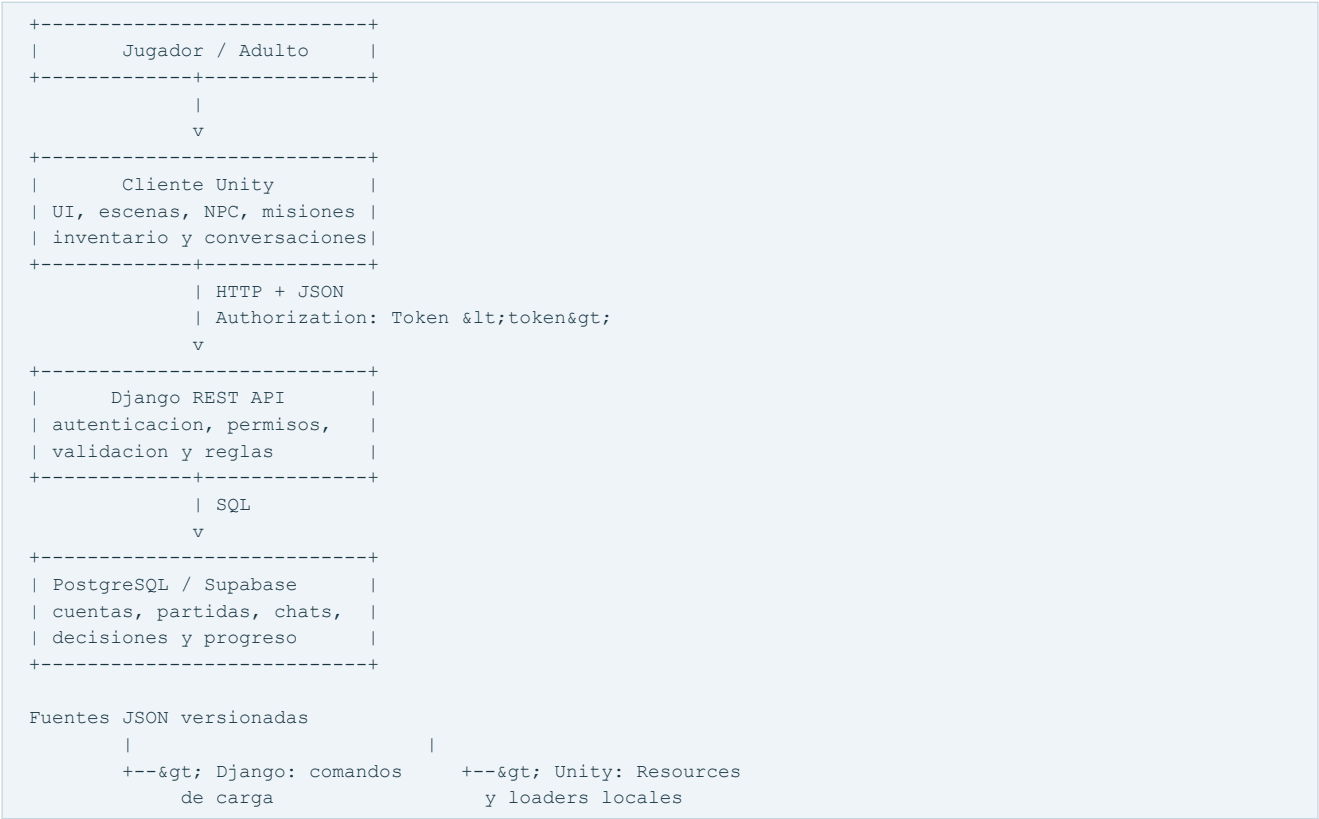
Capa	Tecnología	Responsabilidad
Juego	Unity 6, C# y URP 2D	Interfaz, movimiento, NPC, diálogos, misiones e inventario
API	Django y Django REST Framework	Autenticación, autorización, reglas de negocio y persistencia
Datos	PostgreSQL en Supabase	Cuentas, perfiles, partidas, chats, decisiones y progreso
Contenido	JSON versionado	Preguntas, opciones, diálogos y casos de detective
Pruebas	Django TestCase y Unity Test Framework	Contratos, permisos, guardado y comportamiento de juego

Estado actual

Area	Estado	Observacion
Backend Django	Operativo	La validacion interna y las 45 pruebas automatizadas pasan
Autenticacion por token	Operativa	Registro, login y perfil del adulto disponibles
Perfiles infantiles	Operativos	Crear, listar, editar, borrar y consultar partidas
Partidas y riesgo	Operativos	Creacion, actualizacion y resumen de riesgo por zona
Chats y decisiones	Operativos	Registro de mensajes, preguntas y opciones elegidas
Banco de preguntas	Operativo	19 preguntas interactivas y 34 opciones
Casos de detective	Operativos en API	2 casos y 17 mensajes disponibles
Misiones y album	Implementados en datos	Modelos, migracion, admin y carga de catalogo disponibles
Inventario	Operativo en SampleScene	La UI esta conectada y hay 8 objetos recogibles; parte de la metadata sigue incompleta
Login desde Unity	Operativo	Las escenas de login, menus y juego estan incluidas en Build Settings
Persistencia local Unity	Parcial	Misiones usan PlayerPrefs; inventario solo persiste durante la ejecucion
Conexion Supabase	Dependiente del entorno	Puede requerir Session Pooler o una ruta IPv4 disponible

Arquitectura

La arquitectura evita que Unity se conecte directamente a Supabase. Django es la unica puerta de entrada a la base de datos y aplica autenticacion, pertenencia de recursos y reglas de negocio.



Principios de integracion

- 1 Unity consume exclusivamente endpoints HTTP.
- 2 El token identifica al adulto responsable.
- 3 Cada consulta de perfiles, partidas y progreso valida la pertenencia al adulto autenticado.
- 4 Los identificadores del banco (`pregunta_id` y `opcion_id`) viajan desde Unity y se guardan con la decision.
- 5 Los JSON son datos fuente; no deben editarse de manera independiente en cada copia.

Estructura del repositorio

```
Fishy/
|-- README.md           Documentacion principal
|-- DOCS_JSON_API.md    Contrato JSON y API de conversaciones
|-- FLUJO_CONTROL_PARENTAL.md Flujo adulto, perfiles y partidas
|-- Backend/           Servicio Django REST
|   |-- api/           Modelos, vistas, serializers, URLs y pruebas
|   |-- juego_backend/ Configuracion y rutas principales
|   |-- scripts/       Pruebas globales y utilidades
|   |-- manage.py      Entrada de administracion Django
|   |-- requirements.txt Dependencias Python
|   |-- Dockerfile      Imagen del backend
|   |-- docker-compose.yml Servicio web local
|-- Fishy!/            Proyecto Unity activo
|   |-- Assets/
|       |-- Resources/  Bancos JSON consumidos por Unity
|       |-- Scenes/     Escenas del juego
|       |-- Scripts/    Codigo C# por modulo
|       |-- Prefabs/    NPC reutilizables
|       |-- `-- MissionData/ ScriptableObjects de desafios
|-- Packages/          Dependencias del proyecto
|-- `-- ProjectSettings/ Configuracion de Unity
|-- banco_preguntas/   Fuente editorial del contenido
|   |-- banco_preguntas.json Banco interactivo principal
|   |-- detective_cases.json Casos de detective
|   |-- *.md / *.tex    Especificaciones editoriales
|-- Postman/           Coleccion para probar la API
|-- Assets/            Prototipo C# paralelo, fuera del proyecto Unity activo
|-- deprecated/        Implementaciones antiguas no activas
```

Directorios que pueden causar confusion

- `Fishy!/?` es el proyecto que debe abrirse en Unity Hub porque contiene `Assets`, `Packages` y `ProjectSettings`.
- `Assets/?` en la raiz contiene codigo y pruebas de un prototipo paralelo. Al estar fuera de `Fishy!/?`, Unity no lo compila como parte del juego activo.
- `deprecated/?` conserva codigo historico. No debe usarse como fuente para nuevas implementaciones.
- `Backend/.venv/`, `Library/`, `Temp/`, `Logs/` y otros artefactos locales no forman parte del producto y estan ignorados por Git.

Requisitos

Backend

- Python 3.12 o superior recomendado.
- PostgreSQL accesible o, para desarrollo aislado, SQLite mediante `juego_backend.settings_test`.
- Las dependencias de `Backend/requirements.txt`:
 - Django 5 o superior.
 - Django REST Framework.
 - psycopg2-binary.
 - python-decouple.
 - argon2-cffi.

Cliente Unity

- Unity Editor `6000.4.9f1`.

- Modulos de compilacion de la plataforma objetivo.
- Git LFS recomendado para manejar recursos binarios del proyecto.

Paquetes relevantes declarados en Unity:

- Input System **1.19.0**.
- Universal Render Pipeline **17.4.0**.
- Newtonsoft JSON **3.2.1**.
- Unity Test Framework **1.6.0**.
- UGUI **2.0.0**.
- Paquetes 2D para animacion, sprites y tilemaps.

Herramientas opcionales

- Docker Desktop para ejecutar el backend en contenedor.
- Postman para recorrer la API manualmente.
- Un cliente PostgreSQL para inspeccion administrada de datos.

Inicio rapido

1. Preparar el backend en Windows

Desde PowerShell:

```
cd Backend
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py runserver 127.0.0.1:8000
```

El servicio queda disponible en <http://127.0.0.1:8000/> y la comprobacion publica en <http://127.0.0.1:8000/api/health/>.

El ayudante incluido ofrece el mismo flujo:

```
cd Backend
.\run.ps1
```

Opciones utiles:

```
.\run.ps1 --check
.\run.ps1 --smoke
.\run.ps1 --global
.\run.ps1 --global --fase 3
```

2. Preparar el backend en Linux o macOS

```
cd Backend
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py runserver 127.0.0.1:8000
```

Tambien puede utilizarse [./run.sh](#).

3. Ejecutar sin Supabase

La configuracion de pruebas usa SQLite y permite desarrollar la API sin una conexion remota:


```
cd Backend
python manage.py migrate --settings=juego_backend.settings_test
python manage.py runserver 127.0.0.1:8000 --settings=juego_backend.settings_test
```

La base local se crea en `Backend/test_db.sqlite3` y esta ignorada por Git.

4. Abrir Unity

- 1 Abrir Unity Hub.
- 2 Seleccionar la carpeta `Fishy!` del repositorio.
- 3 Confirmar que el editor usado sea compatible con `6000.4.9f1`.
- 4 Abrir `Assets/Scenes/Boot.unity` o `Assets/Scenes/SampleScene.unity`.
- 5 Verificar que la URL base de `ApiManager` apunte al backend local.
- 6 Iniciar Play Mode.

Configuracion del backend

Variables de entorno

Django lee las siguientes variables:

Variable	Proposito	Valor en desarrollo
<code>DJANGO_SECRET_KEY</code>	Firma criptografica de Django	Una cadena larga y privada
<code>DJANGO_DEBUG</code>	Activa detalles de depuracion	<code>True</code>
<code>DB_NAME</code>	Nombre de la base PostgreSQL	<code>postgres</code>
<code>DB_USER</code>	Usuario de PostgreSQL	Usuario entregado por Supabase
<code>DB_PASSWORD</code>	Contraseña de PostgreSQL	Secreto local
<code>DB_HOST</code>	Host directo o Session Pooler	Host entregado por Supabase
<code>DB_PORT</code>	Puerto PostgreSQL	<code>5432</code> o el indicado por el pooler
<code>DB_CONN_MAX_AGE</code>	Reutilizacion de conexiones en segundos	<code>0</code> en desarrollo
<code>DB_SSLMODE</code>	Modo SSL de PostgreSQL	<code>require</code>

El archivo `.env` nunca debe compartirse ni documentar valores reales. En el estado actual existe un `Backend/.env` versionado y no existe un `.env.example`; antes de publicar o desplegar el repositorio se deben retirar las credenciales del historial, rotarlas y agregar una plantilla sin secretos.

Configuraciones Django

- `juego_backend.settings`: PostgreSQL/Supabase, autenticacion real y ejecucion normal.
- `juego_backend.settings_test`: SQLite local y hasher rapido para pruebas.

Autenticacion y permisos

El backend usa Token Authentication y Session Authentication. Salvo los endpoints publicos de salud, registro y login, la API exige:

```
Authorization: Token <token_del_adulto>
```

El modelo de usuario de Django es `api.AdultoResponsable`. El campo de acceso es `nombre`; el correo tambien es unico. Las contraseñas se almacenan mediante los hashers de Django y no deben guardarse ni enviarse como texto fuera del registro o login protegido por HTTPS.

Base de datos remota

La configuracion normal exige SSL. Si el host directo de Supabase no resuelve o la red no admite IPv6, usar el Session Pooler compatible con IPv4 y actualizar `DB_HOST`, `DB_USER` y `DB_PORT` con los valores que entrega el panel de Supabase.

Configuracion del proyecto Unity

Escenas

Escena	Uso	Incluida actualmente en Build Settings
<code>Boot.unity</code>	Inicializacion y entrada al flujo	Si
<code>SampleScene.unity</code>	Mundo principal y sistemas jugables	Si
<code>Ingresar.unity</code>	Login heredado	Si
<code>MenuUno.unity</code>	Menu heredado	Si
<code>MenuDos.unity</code>	Segundo menu heredado	Si

Las cinco escenas estan incluidas en el estado actual de `dev`. El login heredado carga `MenuDos` por nombre y, si la escena deja de estar disponible, muestra un error controlado en vez de intentar un indice inexistente.

Scripts y modulos activos

Modulo	Componentes principales	Funcion
API	<code>ApiManager</code> , <code>ApiSmokeTest</code>	Solicitudes HTTP, token y comprobacion del backend
Autenticacion/UI	<code>AuthScreen</code> , <code>LoadingScreen</code> , <code>UiBootstrap</code>	Registro, login, perfiles y carga inicial
Movimiento	<code>OttoController</code> , <code>OttoOnScreenButton</code>	Control del personaje e input movil
Mundo	<code>WorldZoneManager</code> , <code>BlockedZone</code> , <code>ZonePopupUI</code>	Zonas, bloqueos y avisos
NPC	<code>NPC</code> , <code>InteractionDetector</code> , <code>NPCDialogue</code>	Deteccion e interaccion contextual
Chat	<code>ChatModuleController</code> , loaders, UI y logger	Conversaciones ramificadas y guardado en backend
Detective	manager, loader, launcher y UI	Casos, mensajes sospechosos y progreso
Misiones	<code>MissionManager</code> , <code>MissionPanelUI</code> , mundo de mision	Catalogo, objetivos y seguimiento
Inventario	<code>InventoryManager</code> , <code>InventoryManagerUI</code> , <code>WorldItem</code>	Recogida y presentacion de objetos
Telefono	<code>OttoPhone</code> , chat launcher y zoom	Acceso diegetico a conversaciones
Menus	controladores de menus y pestanas	Navegacion entre pantallas

Entrada e interaccion

`InteractionDetector` busca objetos que implementan `IInteractable` dentro de un `CircleCollider2D` configurado como trigger. En la escena principal el radio es 1 unidad. La interaccion se activa con:

- Teclado: tecla `E`.
- Gamepad: boton sur.
- Pantalla: controles que llamen al mismo flujo de interaccion.

Configurar la API en Unity

`ApiManager` centraliza la URL base y el token. Para desarrollo local:

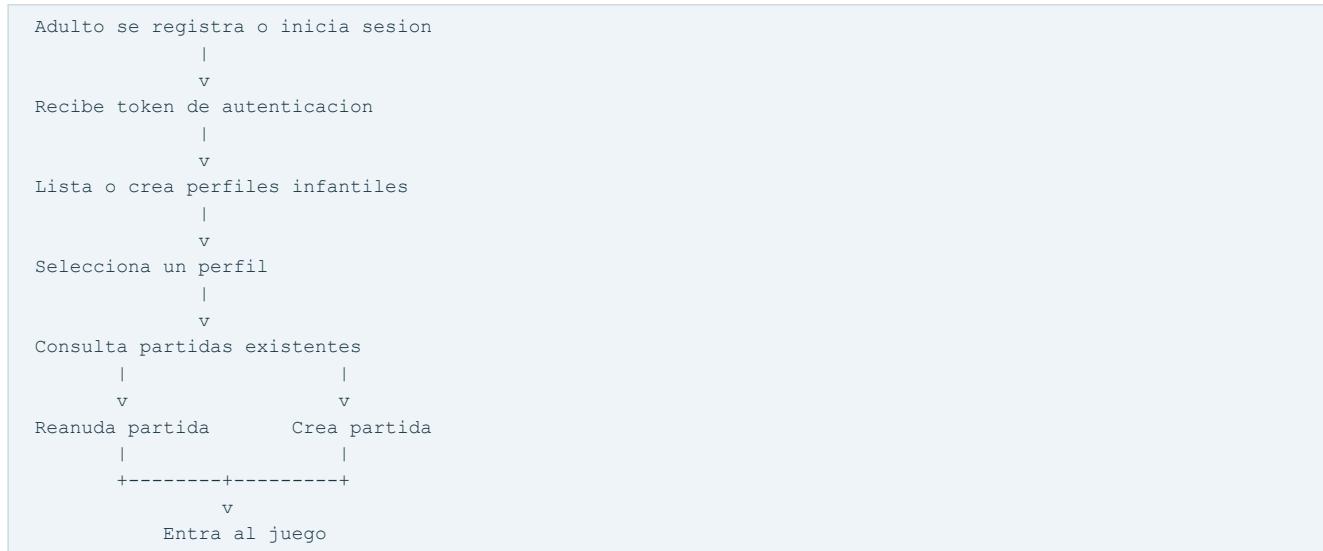
- Editor o juego en el mismo computador: `http://127.0.0.1:8000/api`.

- Telefono fisico en la misma red: `http://<IP-LAN-del-computador>:8000/api` y backend iniciado en `0.0.0.0:8000`.
- Emulador Android: puede requerir la direccion especial del host proporcionada por el emulador.

En dispositivos fisicos, abrir el puerto solo en redes de confianza y no exponer un servidor de desarrollo a Internet.

Flujos principales

Control parental e inicio de partida



El backend filtra todos los perfiles y partidas por el adulto autenticado. Un token valido no autoriza a leer o modificar recursos pertenecientes a otra cuenta.

Conversacion con NPC

- 1 El jugador entra al rango del NPC.
- 2 `InteractionDetector` habilita la interaccion.
- 3 El modulo de chat carga un nodo del banco por zona y contexto.
- 4 Se muestran las opciones disponibles.
- 5 El jugador elige una respuesta.
- 6 Unity aplica la reaccion local de Otto y avanza la conversacion.
- 7 `ChatBackendLogger` registra el mensaje, `pregunta_id` y `opcion_id`.
- 8 Al finalizar se actualiza el resultado del chat y el riesgo asociado.

Caso de detective

- 1 Unity carga el catalogo de casos.
- 2 El jugador revisa una secuencia de mensajes.
- 3 Marca mensajes de riesgo o ambiguos.
- 4 El backend recibe el progreso del caso para la partida.
- 5 La interfaz presenta el avance y permite recuperar los casos de la partida.

Progreso de mision

- 1 Un `MissionGiver` ofrece un `DesafioData`.
- 2 `MissionManager` acepta y mantiene la mision activa.

- 3 Los objetivos notifican avance al tracker.
- 4 `MissionPanelUI` y `QuestPageUI` muestran estado y objetivos.
- 5 La mision completada se registra localmente en `PlayerPrefs`.

Sistemas del juego

Inventario y objetos recogibles

El inventario activo esta formado por:

- `ItemData`: ScriptableObject con nombre, icono y datos del objeto.
- `WorldItem`: componente del objeto presente en el mundo.
- `InventoryManager`: almacenamiento runtime y evento de cambios.
- `InventoryManagerUI`: representacion visual del contenido.

Para convertir un objeto en recogible:

- 1 Crear un asset `ItemData` desde el menu de creacion correspondiente.
- 2 Asignar nombre, icono y demas propiedades.
- 3 Agregar `WorldItem` al GameObject del mundo.
- 4 Asignar el `ItemData` al campo del componente.
- 5 Agregar un `Collider2D` configurado como trigger si la interaccion lo requiere.
- 6 Confirmar que el objeto se encuentra en una capa detectable por `InteractionDetector`.

Al interactuar, `WorldItem` agrega el dato a `InventoryManager` y elimina o desactiva la representacion del mundo.

Para mostrarlo en pantalla, la pagina del inventario debe tener un `InventoryManagerUI`. En `SampleScene`, `InventoryPage` ya contiene ese componente y usa su montaje automatico cuando no se asigna un prefab o contenedor manual. La escena tiene 8 instancias de `WorldItem` con collider y referencia a `ItemData`: brujula, tres flores, mochila, roca, silbato y surf.

La carpeta `Fishy!/Assets/Items/` contiene 9 assets. Brujula y las tres flores tienen nombre e icono configurados. Megafono, mochila, roca, silbato y surf todavia tienen vacios `itemName` e `itemIcon`; la UI usa el texto de respaldo (sin nombre) cuando corresponde. El asset de megafono existe, pero el objeto `megafono` de la escena aun no tiene un `WorldItem` enlazado.

Misiones

`DesafioData` define los desafios como ScriptableObjects. Los assets activos incluyen:

- `Desafio_ChatzonaDesconocidos.asset`.
- `Desafio_DetectiveCaso01.asset`.

`MissionManager` sobrevive a los cambios de escena y utiliza `PlayerPrefs` para conservar el estado local. La carpeta de mision incluye pruebas `PlayMode` y una definicion de assembly independiente.

Riesgo y zonas

La API mantiene un catalogo de niveles de riesgo. Las decisiones de chat y el progreso se asocian a partidas, NPC y zonas. El endpoint de riesgo por zona resume el estado para que Unity pueda actualizar bloqueos, avisos, reacciones o progresion.

Estado y expresiones de Otto

Las opciones del banco pueden incluir una reaccion de Otto. `OttoMoodController` traduce esa reaccion al estado visual o emocional mostrado por el personaje. Los valores deben mantenerse alineados entre el JSON, el loader y los estados configurados en Unity.

Telefono y chat

El telefono funciona como punto de acceso a conversaciones y utiliza sus propios controladores de apertura, chat y zoom. El modulo reutiliza el banco de preguntas y el registro de decisiones del backend.

Banco de contenido

Banco de preguntas

La fuente principal es `banco_preguntas/banco_preguntas.json`, actualmente en version 1.8.

Inventario de contenido:

Elemento	Cantidad
Preguntas interactivas	19
Opciones de respuesta	34
Dialogos neutrales	7
Zonas con preguntas	3

Distribucion de preguntas por zona:

Zona	Preguntas
Desconocidos	12
Ciberacoso	5
Reto viral	2

El banco cubre conversaciones neutrales y categorias de grooming, privacidad, ciberacoso, exclusion y retos virales. Tambien contiene metadatos de version, autor, fechas, historias de usuario y formato esperado de respuesta.

Casos de detective

La fuente `banco_preguntas/detective_cases.json` esta en version 1.0.

Caso	Mensajes	Mensajes de riesgo	Ampliaciones
DC_CASO_01	9	4	1
DC_CASO_02	8	4	1
Total	17	8	2

Fuente de verdad y sincronizacion

`banco_preguntas/banco_preguntas.json` es la fuente editorial del banco interactivo. Unity contiene una copia byte a byte en `Fishy!/Assets/Resources/banco_preguntas.json`; ambas deben mantenerse identicas.

Flujo recomendado al editar contenido:

- 1 Modificar el JSON en `banco_preguntas/`.
- 2 Validar sintaxis, IDs unicos, enlaces entre nodos y formato de opciones.
- 3 Copiar la version validada a `Fishy!/Assets/Resources/`.
- 4 Ejecutar el comando de carga de Django.
- 5 Ejecutar pruebas del backend y pruebas de carga en Unity.
- 6 Confirmar que el numero de version cambio cuando corresponde.

No se recomienda editar primero la copia de Unity, porque puede provocar divergencias entre la experiencia local y los datos persistidos por Django.

Modelo de datos

Identidad y juego

Modelo	Responsabilidad	Relaciones principales
AdultoResponsable	Cuenta autenticada	Tiene muchos perfiles infantiles
UsuarioJugador	Perfil del niño o niña	Pertenece a un adulto y tiene partidas
Partida	Sesion/progreso jugable	Pertenece a un perfil y agrupa el progreso
PersonajeJugador	Estado del personaje	Relacion uno a uno con la partida
NivelRiesgo	Catalogo de riesgo	Referenciado por decisiones y estado

El nombre de un perfil infantil es unico dentro de la cuenta de su adulto, no globalmente.

Conversaciones

Modelo	Responsabilidad
NPC	Instancia del personaje no jugador en una partida
Chat	Conversacion entre partida y NPC
Mensaje	Entrada registrada con pregunta/opcion elegida
PreguntaBanco	Pregunta normalizada del banco
OpcionBanco	Opcion asociada a una pregunta
PosibleRespuesta	Respuesta disponible en el modelo conversacional
Zona	Catalogo y agrupacion territorial/tematica

Detective

Modelo	Responsabilidad
CasoDetective	Definicion del caso
MensajeDetective	Mensaje clasificable dentro del caso
CasoDetectiveProgreso	Estado del caso para una partida

Misiones y album

Modelo	Responsabilidad
Mision	Catalogo persistente de misiones
DialogoNPC	Dialogos ligados al catalogo
RecompensaAlbum	Recompensa coleccionable disponible
RecompensaObtenida	Union entre partida y recompensa obtenida

Los modelos de catalogo existen en la base y en el administrador, aunque el estado actual de las rutas publicas no expone endpoints REST dedicados para todas las operaciones de misiones y album.

Migraciones

Las migraciones vigentes son:

- 1 0001_initial.py: esquema base.
- 2 0002_mensaje_opcion_banco_id.py: identificador de opcion del banco en mensajes.

3 0003_detective.py: modelos del modulo detective.

4 0004_catalogo_misiones_album.py: misiones, dialogos y recompensas.

No existe una migracion 6 en la linea actual. El intento relacionado con login de Google fue retirado y no forma parte del esquema soportado.

API HTTP

Todas las rutas se encuentran bajo `/api/`. La administracion de Django se encuentra en `/admin/`.

Autenticacion

Método	Ruta	Acceso	Funcion
GET	<code>/api/health/</code>	Publico	Comprueba que el servicio responde
POST	<code>/api/auth/registro/</code>	Publico	Crea un adulto responsable y token
POST	<code>/api/auth/login/</code>	Publico	Valida credenciales y entrega token
GET	<code>/api/auth/perfil/</code>	Token	Devuelve el adulto autenticado

Ejemplo de login:

```
POST /api/auth/login/
Content-Type: application/json

{
  "nombre": "adulto_demo",
  "password": "contrasena_segura"
}
```

Perfiles infantiles

Método	Ruta	Funcion
GET	<code>/api/jugadores/</code>	Lista los perfiles del adulto
POST	<code>/api/jugadores/</code>	Crea un perfil infantil
GET	<code>/api/jugadores/<jugador_id>/</code>	Obtiene un perfil propio
PATCH	<code>/api/jugadores/<jugador_id>/</code>	Actualiza parcialmente el perfil
DELETE	<code>/api/jugadores/<jugador_id>/</code>	Elimina un perfil propio
GET	<code>/api/jugadores/<jugador_id>/partidas/</code>	Lista sus partidas

Partidas, NPC y riesgo

Método	Ruta	Funcion
GET	<code>/api/niveles-riesgo/</code>	Lista el catalogo de niveles
POST	<code>/api/partidas/</code>	Crea una partida
GET	<code>/api/partidas/<partida_id>/</code>	Recupera una partida propia
PATCH	<code>/api/partidas/<partida_id>/</code>	Actualiza una partida propia
GET	<code>/api/partidas/<partida_id>/npcs/</code>	Lista NPC de la partida
POST	<code>/api/partidas/<partida_id>/npcs/</code>	Crea un NPC asociado
GET	<code>/api/partidas/<partida_id>/riesgo-por-zona/</code>	Resume el riesgo por zona
PATCH	<code>/api/npcs/<npc_id>/</code>	Actualiza un NPC propio

Chats y mensajes

Método	Ruta	Función
POST	/api/chats/	Inicia un chat
GET	/api/chats/<chat_id>/mensajes/	Lista mensajes registrados
POST	/api/chats/<chat_id>/mensajes/registrar/	Registra una decision
POST	/api/chats/<chat_id>/finalizar/	Finaliza y consolida el chat

Ejemplo conceptual de registro de decision:

```
{
  "pregunta_id": "PREGUNTA_DESCONOCIDOS_01",
  "opcion_id": "OPCION_A",
  "contenido": "Respuesta elegida por el jugador"
}
```

Los campos exactos y sus aliases de compatibilidad se validan en los serializers y en las pruebas de contrato. Consultar [DOCS_JSON_API.md](#) antes de modificar el formato.

Banco de preguntas

Método	Ruta	Función
GET	/api/banco/zonas/	Lista las zonas cargadas
GET	/api/banco/zonas/<zona>/preguntas/	Preguntas de una zona
GET	/api/banco/preguntas/	Lista el banco normalizado
GET	/api/banco/preguntas/<pregunta_id>/	Obtiene una pregunta y opciones

Detective

Método	Ruta	Función
GET	/api/casos-detective/	Lista casos disponibles
GET	/api/casos-detective/<caso_id>/	Obtiene caso y mensajes
POST	/api/casos-detective/<caso_id>/progreso/	Guarda el progreso del caso
GET	/api/partidas/<partida_id>/casos-detective/	Lista progreso de la partida

Codigos de respuesta esperados

Código	Significado habitual
200	Consulta o actualizacion exitosa
201	Recurso creado
204	Eliminacion exitosa sin cuerpo
400	Datos incompletos o invalidos
401	Token ausente o invalido
403	Accion no permitida
404	Recurso inexistente o ajeno al adulto
500	Error no controlado del servidor

Por seguridad, algunos recursos ajenos pueden responder 404 en vez de revelar que existen.

Persistencia

Dato	Lugar	Duracion
Cuenta del adulto	PostgreSQL/Supabase	Permanente
Perfil infantil	PostgreSQL/Supabase	Permanente
Partida y personaje	PostgreSQL/Supabase	Permanente
Chats y decisiones	PostgreSQL/Supabase	Permanente
Progreso detective	PostgreSQL/Supabase	Permanente
Catalogos de mision/album	PostgreSQL/Supabase	Permanente
Estado local de misiones Unity	PlayerPrefs	Entre ejecuciones en el dispositivo
Inventario Unity	Memoria y DontDestroyOnLoad	Entre escenas, no entre reinicios
Token Unity	Estado administrado por el cliente	Depende de la implementacion de sesion

El inventario actual no se envia al backend. Si debe sobrevivir al cierre del juego o sincronizarse entre dispositivos, se necesita un modelo persistente, endpoints con validacion de pertenencia y serializacion desde Unity.

Pruebas y validacion

Backend

La suite contiene 45 pruebas automatizadas y, en el estado documentado, todas pasan. Cubre:

- Administracion Django.
- Carga del banco JSON.
- Catalogo de misiones y recompensas.
- Flujo existente de autenticacion, perfiles y partidas.
- Guardado de decisiones y contratos de datos.
- Casos de detective.
- Permisos y aislamiento entre adultos.

Ejecutar:

```
cd Backend
python manage.py check --settings=juego_backend.settings_test
python manage.py makemigrations --check --dry-run --settings=juego_backend.settings_test
python manage.py test --settings=juego_backend.settings_test
```

Prueba de humo

Con el servidor activo:

```
cd Backend
python scripts/smoke_test.py
```

Tambien puede usarse `run.ps1 --smoke`.

Prueba global

`scripts/test_global.py` organiza la validacion en siete fases:

- 1 Preflight del entorno.
- 2 Pruebas unitarias.
- 3 API y contratos HTTP.

- 4 Banco de preguntas y ORM.
- 5 Casos de detective.
- 6 Contrato de DTO con Unity.
- 7 Compilacion Unity opcional.

Ejemplos:

```
.\run.ps1 --global  
.\run.ps1 --global --fase 6
```

Unity

El modulo de misiones contiene pruebas PlayMode para aceptacion, avance y finalizacion. Ademas de las pruebas automatizadas, antes de entregar una version se recomienda comprobar manualmente:

- Boot, registro, login y seleccion de perfil.
- Creacion y reanudacion de partida.
- Interaccion con NPC usando teclado, gamepad y controles tactiles.
- Conversacion completa y persistencia en el backend.
- Caso de detective completo.
- Cambio de escena sin perder managers persistentes.
- Recogida de un objeto y refresco visual del inventario.

Carga y administracion de datos

Cargar banco de preguntas y catalogos

Desde **Backend/**:

```
python manage.py cargar_banco
```

El comando normaliza el banco y mantiene los catalogos correspondientes. La carga debe ejecutarse despues de aplicar migraciones y antes de probar los endpoints del banco en una base nueva.

Cargar casos de detective

```
python manage.py cargar_detective
```

Verificar el banco

```
python scripts/verificar_carga_banco.py
```

Administrador Django

Crear un superusuario:

```
python manage.py createsuperuser
```

Luego abrir <http://127.0.0.1:8000/admin/>. El administrador permite inspeccionar cuentas, perfiles, partidas, conversaciones, banco, detective, misiones y recompensas. No sustituye la validacion de permisos de la API y no debe exponerse sin HTTPS, restricciones de red y credenciales robustas.

Postman

La carpeta **Postman/** contiene una coleccion para recorrer el flujo HTTP. Configurar la URL base, ejecutar registro o login y reutilizar el token en las solicitudes protegidas.

Despliegue y seguridad

Docker

El backend incluye `Dockerfile` y `docker-compose.yml`:

```
cd Backend
docker compose up --build
```

El compose levanta el servicio web, pero no crea una base PostgreSQL local: espera conectarse al PostgreSQL configurado. Los comandos que leen archivos del banco necesitan que esos archivos esten incluidos o montados dentro del contenedor.

Antes de produccion

- Usar un `DJANGO_SECRET_KEY` unico y privado.
- Desactivar `DJANGO_DEBUG`.
- Reemplazar `ALLOWED_HOSTS = ["*"]` por dominios conocidos.
- Servir exclusivamente mediante HTTPS.
- Rotar cualquier credencial que haya estado versionada.
- Sacar `.env` del control de versiones y agregar una plantilla segura.
- Restringir el administrador Django.
- Configurar CORS solo si el cliente final lo necesita.
- Aplicar migraciones como paso controlado del despliegue.
- Ejecutar pruebas y comprobaciones antes de cada publicacion.
- Definir copias de seguridad y restauracion de PostgreSQL.
- Establecer logs sin contraseñas, tokens ni datos sensibles de menores.
- Revisar retencion, consentimiento y acceso a datos infantiles segun la normativa aplicable.

Consideraciones sobre menores

El proyecto trata datos de cuentas adultas y perfiles infantiles. La puesta en produccion debe minimizar los datos personales, documentar consentimiento y retencion, permitir eliminacion verificable, proteger tokens y evitar incluir texto sensible en logs o analíticas.

Problemas conocidos

- 1 El inventario solo persiste en memoria durante la ejecucion.
- 2 Cinco `ItemData` no tienen nombre ni icono; se muestran con el texto de respaldo. Megafono tampoco esta enlazado como `WorldItem` en `SampleScene`.
- 3 La conexion directa a Supabase puede fallar en redes sin resolucion/ruta IPv6; el Session Pooler suele ser la alternativa.
- 4 `Backend/.env` esta versionado y no existe `.env.example`; es una deuda de seguridad que debe corregirse antes de compartir el repositorio.
- 5 `ALLOWED_HOSTS` permite cualquier host y los valores por defecto de desarrollo no son adecuados para produccion.
- 6 El directorio raiz `Assets/` puede confundirse con el proyecto Unity, pero es un prototipo paralelo.
- 7 Una declaracion de atributos Git dentro de `Fishy!/.gitattributes` puede producir advertencias sobre macros; esto no afecta la ejecucion, pero conviene consolidarla en la raiz.
- 8 Los modelos de misiones y album no tienen cobertura completa de endpoints publicos para sincronizar todo el progreso desde Unity.

Solucion de problemas

El backend no inicia por conexion a PostgreSQL

- Confirmar `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` y `DB_PORT`.
- Probar el Session Pooler de Supabase si falla el host directo.
- Confirmar que `DB_SSLMODE=require`.
- Para seguir desarrollando sin red, iniciar con `juego_backend.settings_test`.

`/api/health/` no responde

- Confirmar que el entorno virtual esta activo.
- Verificar que las dependencias esten instaladas.
- Ejecutar `python manage.py check`.
- Revisar que otro proceso no este ocupando el puerto 8000.
- Probar `http://127.0.0.1:8000/api/health/`, incluyendo el prefijo `/api`.

Unity no llega al backend

- Confirmar la URL base de `ApiManager`.
- Desde un telefono, no usar `127.0.0.1`; usar la IP LAN del computador.
- Iniciar Django en `0.0.0.0:8000` solo dentro de una red confiable.
- Revisar firewall, puerto y que ambos dispositivos esten en la misma red.
- Confirmar que la respuesta de salud funciona antes de probar login.

Login devuelve 401

- El acceso usa `nombre` y `password`, no necesariamente correo.
- Verificar que el usuario fue creado en la misma base a la que apunta el servidor.
- Borrar cualquier token antiguo del cliente y autenticar nuevamente.
- No enviar el prefijo `Bearer`; la API espera `Authorization: Token ...`.

Error de indice al entrar al login o menu

Este error fue corregido en `dev` al registrar las cinco escenas y eliminar el fallback inseguro al indice 2. Si reaparece, confirmar que la copia local incluya el commit `fee4a09` o uno posterior y que `MenuDos` siga habilitada en Build Settings.

El objeto se recoge, pero no aparece en el inventario

- Confirmar en consola que `WorldItem` encontro un `InventoryManager`.
- Agregar `InventoryManagerUI` a `InventoryPage`.
- Asignar contenedor, prefab visual y referencias requeridas.
- Confirmar que la UI se suscribe al evento de cambio y hace un refresco inicial.
- Verificar que el `ItemData` tenga nombre e icono asignados.

El banco de Unity y el backend muestran contenido distinto

- Comparar `banco_preguntas/banco_preguntas.json` con `Fishy!/Assets/Resources/banco_preguntas.json`.
- Volver a copiar desde la fuente editorial.

- Ejecutar `cargar_banco` en la base activa.
- Limpiar datos cacheados o reiniciar Play Mode si el loader ya cargo la version anterior.

Las migraciones no estan sincronizadas

```
python manage.py makemigrations --check --dry-run
python manage.py showmigrations
python manage.py migrate
```

No crear migraciones vacias para ocultar diferencias. Primero identificar si el cambio de modelo es intencional.

Flujo de trabajo recomendado

Ramas

- 1 Actualizar `dev` desde el remoto.
- 2 Crear una rama `codex/<descripcion>` basada directamente en `dev`.
- 3 Mantener los cambios acotados a una funcion o correccion.
- 4 Ejecutar pruebas proporcionales al cambio.
- 5 Revisar que no se incluyan secretos, bases locales ni artefactos de Unity.
- 6 Integrar en `dev` solo despues de validar el flujo funcional.

Cambios de backend

- 1 Modificar modelos, serializers, vistas o rutas.
- 2 Crear migracion solo si cambia el esquema.
- 3 Ejecutar `check`, comprobacion de migraciones y suite completa.
- 4 Actualizar Postman y esta documentacion si cambia el contrato.

Cambios de contenido

- 1 Editar la fuente en `banco_preguntas/`.
- 2 Validar el JSON.
- 3 Sincronizar la copia de Unity.
- 4 Cargar en Django.
- 5 Probar el recorrido completo y los IDs guardados.

Cambios de Unity

- 1 Abrir `Fishy!` como proyecto.
- 2 Evitar editar escenas no relacionadas para reducir conflictos YAML.
- 3 Conservar archivos `.meta` junto a sus assets.
- 4 Ejecutar las pruebas disponibles y un recorrido manual.
- 5 Verificar Build Settings si se agregan o renombran escenas.

Documentacion complementaria

- [Backend/README.md](#) (Backend/README.md): instalacion y operacion detallada del backend.
- [DOCS_JSON_API.md](#) (DOCS_JSON_API.md): contrato entre el banco JSON, Unity y Django.
- [FLUJO_CONTROL_PARENTAL.md](#) (FLUJO_CONTROL_PARENTAL.md): recorrido de cuentas, perfiles y partidas.

- [banco_preguntas/BANCO_PREGUNTAS.md](#) (banco_preguntas/BANCO_PREGUNTAS.md): catalogo editorial del banco.
- [banco_preguntas/FLUJOS_CONVERSACIONALES.md](#) (banco_preguntas/FLUJOS_CONVERSACIONALES.md): estructura de dialogos y transiciones.
- [Fishy!/README.md](#) (Fishy!/README.md): notas operativas del proyecto Unity.
- [Postman/Fishy_API.postman_collection.json](#) (Postman/Fishy_API.postman_collection.json): coleccion de solicitudes HTTP.

Mantenimiento de este documento

Actualizar este README cuando cambie cualquiera de los siguientes elementos:

- Variables de entorno o proceso de arranque.
- Escenas incluidas en la compilacion.
- Rutas, payloads o permisos de la API.
- Modelos o migraciones.
- Formato o version de los bancos JSON.
- Estrategia de persistencia de inventario, misiones o token.
- Cantidad o alcance de pruebas.
- Requisitos de despliegue y seguridad.

El PDF entregado junto a esta documentacion se genera a partir de este mismo archivo. El README es la fuente editable y debe regenerarse el PDF despues de cada cambio relevante.